

异或空间线性基

在线性代数中，基是描述向量空间的基本工具。向量空间的基是这个向量空间的一个特殊子集，基的元素称为基向量，基向量相互之间线性无关。向量空间的任意元素都能唯一地用基向量的线性组合来表示。一个向量空间的基可以有很多，但是每个基的基向量个数相同（最小数量），由此可以把大问题“压缩”为小问题（基）提高效率。

异或空间线性基的概念

线性空间和基的概念可以推广应用于异或空间，帮助高效率地求解异或问题。异或空间线性基是利用了二进制特征的一个数学“小游戏”，借助高斯消元求线性基是一种清晰易懂的方法。

1. 线性基的概念

用下面的例题引出异或空间线性基的概念。

线性基

问题描述：给定 n 个整数（数字可能有重复），在这些数中选取任意个，使它们的异或和最大。二进制的异或（XOR，用符号 $\wedge$ 表示）计算规则： $1\wedge 1 = 0$, $1\wedge 0 = 1$, $0\wedge 1 = 1$, $0\wedge 0 = 0$ 。两个数异或时，先转换为二进制，然后再异或，如 $2\wedge 3 = 10_2\wedge 11_2 = 01_2 = 1$ 。给出 N 个数，取其中任意个数进行异或计算，得到一个集合。例如，给出3个数 $\{2, 3, 4\}$ ，取其中任意个数作异或可得 $\{2, 3, 4, 2\wedge 3, 2\wedge 4, 3\wedge 4, 2\wedge 3\wedge 4\} = \{2, 3, 4, 1, 6, 7, 5\}$ 。其中最大的异或和是 $3\wedge 4 = 7$ 。

输入：第1行输入一个整数 n ，表示元素个数，下面一行输入 n 个数。

输出：输出一个数，表示答案。

“取其中任意个数”，如果简单地组合出所有情况， n 个数的任意组合共有 2^n 种，本题 $n = 10000$ ，不可能计算出来。

不过，虽然有 2^n 种组合，但是所有组合的异或结果（值域）却少得多。设 n 个数中最大的数有 m 位，二进制为10001000111100，共14位，则 $m = 14$ 。原因很简单：任意两个数 a 、 b 作异或计算 $c = a\wedge b$ ， c 的位数不大于 a 、 b 中最大的位数。所以， 2^m 种组合的异或结果为 $0 \sim 2^m - 1$ ，最多有 2^m 种。例如，设 n 个数字都是64位的long long型，其中最大的数有 $m = 63$ 位，那么异或的计算结果最多有 2^{63} 种。

“ n 个物品的任意组合共有 2^n 种”，虽然可以用组合公式来证明，但可以很简单地用二进制推理，用一个 n 位的二进制数表达组合情况，每个二进制数表示一种组合情况。以 $n = 3$ 个物品为例，用二进制数101表示取了第1个和第3个物品，000~111共有 2^3 种组合情况。理解了这种思路，就容易理解本节的内容。

有没有可能把 2^m 种组合缩小到 m 种组合？这就是用线性基求解异或问题的思路：把对 n 个数的组合求异或，缩小到对 m 个数（线性基）的组合求异或，从而把 2^n 规模的问题缩小到 2^m 规模的问题。设原数字的集合为 $A = \{a_1, a_2, \dots, a_n\}$ ，求得线性基 $P = \{p_1, p_2, \dots, p_k\}$ 。在 A 和 P 上分别对任意组合求异或，结果是一样的，换句话说，对 A 异或与对 P 异或是等价的。注意，这里的“等价”不包括0， A 的异或计算可能有0，而 P 的异或计算没有0。

例如， $A = \{2, 3, 5, 6, 7\}$ ， $n = 5$ ，它的任意组合有 $2^5 - 1 = 31$ 种，注意这里不是 2^5 ，去掉了空集的情况。对 A 中元素的所有组合作异或计算，结果是 $\{0, 1, 2, 3, 4, 5, 6, 7\}$ 。 A 中最大的数是 $k = 3$ 位，用后面的方法计算得到线性基 $P = \{5, 2, 1\}$ ，有 $2^3 - 1 = 7$ 种组合，异或结果也是 $\{1, 2, 3, 4, 5, 6, 7\}$ ，正好7种。注意，线性基不是唯一的，如 $A = \{2, 3, 5, 6, 7\}$ ，还有 $\{7, 2, 1\}$ 、 $\{7, 3, 1\}$ 、 $\{5, 2, 1\}$ 等，它们的个数都是3个。

2. 异或空间线性基的性质

异或空间线性基P具有以下性质。

(1) **等价性**：在原数组A上进行异或计算，与在线性基P上进行异或计算结果相同。

(2) **最小性 (最优性)**：P是满足性质 (1) 的所有集合中元素最少的。最小性隐含了线性无关性。即P中不同的组合，其异或结果也不同。

以上两条是线性基的基本要求。从性质(2)可以推出性质(3)。

(3) **线性基P中不存在异或和为0的子集**。如果存在异或和为0的子集，如 $p_a \wedge p_b \wedge p_c \wedge p_x = 0$ ，则 $p_a \wedge p_b \wedge p_c = p_x$ ， p_x 是多余的，与性质(2)矛盾。

但是，P中不存在异或为0的子集，A中却可能有。例如， $A = \{1, 2, 3\}$ ， $P = \{1, 2\}$ ， $1 \wedge 2 \wedge 3 = 0$ 。这实际上也是 $1 \wedge 2 = 3$ ，3是多余的。

构造线性基P时遵循一个规则：**P中每个元素的二进制位数均不相同**。这实际上也是性质(2)的要求。P中元素的最少位数为1，最大位数为m，那么P的所有元素个数不会超过m个，这保证了P的最小性。而且，这个规则也保证了P的所有元素都是线性无关的，满足这个规则的P的任意组合的异或和都不会相同。进一步推导出，P能组合出的异或和有 $2^k - 1$ 种，不包括空集，其中k为P的元素个数。

接下来的构造过程将说明这个规则是完全能做到的。

线性基的构造

1. 基本原理

如何计算出集合A的线性基P？分析两种情况。

(1) 首先分析一个特例：若A中所有元素的二进制位数都不同，那么A就是它自己的一个线性基。例如， $A = \{1, 3, 9\}$ ，它的一个线性基是它自己 $P = \{1, 3, 9\}$ 。A也有其他线性基，如 $\{1, 2, 9\}$ 、 $\{1, 2, 8\}$ 等。

A的3个元素 $\{1, 3, 9\}$ 的二进制是 $\{1, 11, 1001\}$ ，长度分别为1位、2位、4位。构造线性基P时，P中应该有一个4位的元素，否则无法通过其他不同长度的元素异或出一个第4位为1的数1001；同理，P中也需要一个2位、1位的元素，否则无法通过其他不同长度的元素异或出来。既然如此，直接把A看成自己的线性基即可。

(2) 另一种情况是A中有位数相同的元素。前面已经提到，为满足线性基的最小性，构造P的规则是“P中每个元素的二进制位数均不相同”。在A中把相同位数的元素选一个(如第1个)直接复制到P中，相同位数的其他元素不能再放入P中，如何处理？

先讨论有两个相同位数元素的线性基构造。设 $A = \{a_1, a_2\}$ ，且两个元素位数相同，它的一个线性基是 $P = \{a_1, a_1 \wedge a_2\}$ 。下面证明P与A在异或空间等价。 $a_1 \wedge a_2$ 比 a_1 、 a_2 的长度短，因为 a_1 、 a_2 的最高位被异或计算去掉了。 $\{a_1, a_2\}$ 与 $\{a_1, a_1 \wedge a_2\}$ 的异或计算结果相同，因为 $\{a_1, a_2\}$ 的异或组合是 $\{a_1, a_2, a_1 \wedge a_2\}$ ，而 $\{a_1, a_1 \wedge a_2\}$ 的组合是 $\{a_1, a_1 \wedge a_2, a_1 \wedge a_1 \wedge a_2\} = \{a_1, a_1 \wedge a_2, 0 \wedge a_2\} = \{a_1, a_1 \wedge a_2, a_2\}$ 。

这样就解决了A中有两个相同位数元素的问题。当A中相同位数的元素超过两个时，连续处理即可。例如， $A = \{8, 13, 15\}$ ，用二进制表示为 $A = \{1000, 1101, 1111\}$ ，都有4位，处理过程如下。

① 把第1个数1000放入P中， $P = \{1000\}$ 。

② 第2个数1101与P中的1000异或， $1101 \wedge 1000 = 101$ ，放入P中， $P = \{1000, 101\}$ 。

③ 第3个数1111与P中的1000异或，得111；继续与P中的101异或， $111 \wedge 101 = 10$ ，得10，放入P中， $P = \{1000, 101, 10\}$ 。计算结果，线性基用十进制表示为 $P = \{8, 5, 2\}$ 。

分析构造线性基的计算复杂度：A中有n个数，每个数最多与当前P的每个元素异或一次，P最多有m个元素，总复杂度为 $O(nm)$ 。

最后，复述前面提到的一个结果：P能组合出的异或和有 $2^k - 1$ 种，不包括空集，其中k为P的元素个数。

2. 高斯消元法求线性基

上述构造线性基的过程，如果用高斯消元法解释会更清晰，而且借助高斯消元法也能更透彻地理解线性基应用问题。

设原集合 $A = \{a_1, a_2, \dots, a_n\}$ ，最大位数为m，求线性基 $P = \{p_1, p_2, \dots, p_k\}$ ，其中 $p_1 > p_2 > \dots > p_k$ 。

把A的每个整数看作m位二进制数，写成一个n行×m列的0/1矩阵，矩阵的第i行，从左到右依次是 a_i 的第 $m-1, m-2, \dots, 1, 0$ 位。把这个矩阵看作异或方程组的系数矩阵，用高斯消元法求解异或方程，最后把它转换为简化阶梯矩阵，简化阶梯矩阵的非零整行就是A的线性基。简化阶梯矩阵中包含一个单位矩阵，即只有对角线上是1。

其正确性讨论如下。

- (1) 初始系数矩阵的各行的任意组合对应了A中元素的任意组合。
- (2) 初始系数矩阵能变换得到简化阶梯矩阵；反过来，简化阶梯矩阵也能变换得到初始系数矩阵，两者是等价的。
- (3) 简化阶梯矩阵的每行的位数不同，符合线性基的要求，它是一个线性基。

求 $A = \{8, 13, 15\}$ 的线性基。高斯消元过程如下。

$$\begin{bmatrix} 1000 \\ 1101 \\ 1111 \end{bmatrix} \Rightarrow \begin{bmatrix} 1000 \\ 0101 \\ 0111 \end{bmatrix} \Rightarrow \begin{bmatrix} 1000 \\ 0101 \\ 0010 \end{bmatrix}$$

在第1个矩阵上做第1次消元：第1行与第2、3行异或，得中间的矩阵。在中间矩阵上做第2次消元：第2行与第3行异或，得最后一个简化阶梯矩阵，即线性基 $P = \{8, 5, 2\}$ 。

求 $A = \{2, 3, 5, 6, 7\}$ 的线性基。下面概要给出高斯消元过程，最后一个矩阵是简化阶梯矩阵，前3行是一个单位矩阵。

$$\begin{bmatrix} 010 \\ 011 \\ 101 \\ 110 \\ 111 \end{bmatrix} \Rightarrow \begin{bmatrix} 111 \\ 010 \\ 011 \\ 101 \\ 110 \end{bmatrix} \Rightarrow \begin{bmatrix} 111 \\ 010 \\ 001 \\ 000 \\ 000 \end{bmatrix} \Rightarrow \begin{bmatrix} 100 \\ 010 \\ 001 \\ 000 \\ 000 \end{bmatrix}$$

注意简化阶梯矩阵的前3列是一个单位矩阵。下面概要给出高斯消元过程，最后一个矩阵是简化阶梯矩阵，前3行是一个单位矩阵。在简化阶梯矩阵中有两个全为0的行，这说明初始矩阵可以组合出0，即A中有等于0的异或和，如 $3 \wedge 5 \wedge 6 = 0$ 。

线性基的应用

得到线性基后，能高效率处理以下异或问题。

1. 最小异或和

由前面的高斯消元示例可知，若简化阶梯矩阵中有全0的行，说明A的最小异或和为0。

除了0以外，A的最小异或和就是P的最小元素，即位数最小的元素。因为这个元素与P的其他元素异或，必然会增大。由于最小元素只有一个值，所以P中最小的元素是唯一的。

2. 最大异或和

从大到小对P的所有元素运用贪心法，若异或某个元素结果变大，则异或它，否则忽略它。为什么这样操作是对的？以最大元素为例，它必然用在最大异或结果的计算中，因为它的位数比其他元素都多，无论其他元素如何组合，其异或结果的位数都不会大于最大元素的位数，所以它必须选。再以第2大元素为例，如果它能使异或结果变大，就使用它，因为无论其他所有元素如何组合，异或结果都不会超过它。

若使用高斯消元求得的简化阶梯矩阵计算最大异或和，则更简单，直接异或P中所有元素即可。因为只有对角线上是1，这一行所表示的元素必须选中作异或，否则在最后的异或结果中这一位就是0了。

下面是洛谷P3812模板题的代码，求最大异或和。Insert()函数逐个加入A的元素，计算线性基P。这段代码没有用高斯消元，生成的线性基不是形如简化阶梯矩阵的。高斯消元需要离线操作，即读完A的所有元素后才能建立异或方程组。而用Insert()函数可以在线操作，即读完一个数就插入线性基。

数组P[]存储生成的线性基，P[i]表示最高位在第i位的元素。例如，生成用二进制表示的线性基 $P = \{1, 11, 1001\}$ ， $P[3] = 1001$ ， $P[1] = 11$ ， $P[0] = 1$ 。

```
1 //不用高斯消元求线性基
2 #include <bits/stdc++.h>
3 using namespace std;
4 typedef long long ll;
5 const int M = 63;
6 ll p[M]; //线性基
7 bool zero;
8 void Insert(ll x){
9     for(int i = M; i >= 0; i--){
10         if(x >> i & 1){ //x的最高位
11             if(p[i] == 0) { p[i] = x; return; } //p[i]还没有，直接让p[i] = x
12             else x ^= p[i]; //p[i]已经有了，逐个异或
13         }
14         zero = true; //有异或和为0的组合
15     }
16     ll qmax(){
17         ll ans = 0;
18         for(int i = M; i >= 0; i--){
19             ans = max(ans, ans ^ p[i]);
20         }
21         return ans;
22     }
23     int main(){
24         int n; scanf("%d", &n);
25         for(int i = 1; i <= n; i++){
26             ll x; scanf("%lld", &x);
27             Insert(x);
28         }
29         printf("%lld\n", qmax());
30         return 0;
31     }
```

3. 第k大异或和/第k小异或和

如果利用高斯消元得到的简化阶梯矩阵，求第k大异或和问题变得极为简单。例如，简化阶梯矩阵表示的一个线性基P，共有4个元素，每行表示一个元素，如下。

$$\begin{bmatrix} 10001 \\ 01000 \\ 00101 \\ 00011 \end{bmatrix}$$

最大异或和：异或P所有的元素。4行表示的4个元素全都异或，用1111表示全选。

第2大异或和：异或前3行，用1110表示选前3行。

第3大异或和：用1101表示选第1、2、4行的3个元素。

...

综上所述，设P有t个元素，即简化阶梯矩阵有t行，第k大异或和就是选 $2^t - k$ 的二进制对应为1的那些行。例如上面的例子， $k = 1$ ，选 $2^4 - 1 = 15 = 1111_2$ ； $k = 2$ ，选 $2^4 - 2 = 14 = 1110_2$ ；等等。

XOR

问题描述：二进制数的异或（XOR，用符号 $\wedge$ 表示）计算规则： $1 \wedge 1 = 0$ ， $1 \wedge 0 = 1$ ， $0 \wedge 1 = 1$ ， $0 \wedge 0 = 0$ 。两个数异或时，先转换为二进制，然后再异或，如 $2 \wedge 3 = 10_2 \wedge 11_2 = 01_2 = 1$ 。给出N个数，取其中任意个数进行异或计算，得到一个集合。例如，给出3个数 $\{2, 3, 4\}$ ，取其中任意个数作异或可得 $\{2, 3, 4, 2 \wedge 3, 2 \wedge 4, 3 \wedge 4, 2 \wedge 3 \wedge 4\} = \{2, 3, 4, 1, 6, 7, 5\}$ 。

输入：输入N个数，Q个查询。第k个 Q_k 表示查询第k小异或和。 $1 \leq N \leq 10000$ ， $1 \leq Q \leq 10000$ ，所有数 $\leq 10^{18}$ 。

输出：对每个查询，输出结果。若不存在第k小异或和，则输出-1。

Gauss()函数用高斯消元法求线性基，仍然存在a[]数组中。Query(k)函数求第k小异或和。

```
1 //用高斯消元求线性基
2 #include<bits/stdc++.h>
3 #define N 10100
4 using namespace std;
5 typedef long long ll;
6 int n;
7 bool zero; //消元后是否产生全0的行
8 ll a[N];
9 void Gauss(){ //高斯消元求线性基
10     int i,k=1; //k标记当前第几行
11     ll j = (1ll)<<62; //注意不是63
12     for(;j;j>>=1){
13         for(i=k;i<=n;i++){
14             if(a[i]&j) break; //找到第j位是1的a[]
15             if(i > n) continue; //没有第j位是1的a[]
16             swap(a[i],a[k]); //把这一行换到上面
17             for(i=1;i<=n;i++){ //生成简化阶梯矩阵
18                 if(i != k && a[i]&j) a[i]^=a[k];
19             }
20             k++;
21     }
22     k--;
```

```

22     if(k!=n) zero = true;
23     else zero = false;
24     n = k; //线性基中元素的个数
25 }
26 ll Query(ll k){ //第k小异或和
27     ll ans=0;
28     if(zero) k--;
29     if(!k) return 0;
30     for(int i=n;i;i--){
31         if(k&1) ans^=a[i];
32         k >>= 1;
33     }
34     if(k) return -1;
35     return ans;
36 }
37 int main(){
38     int cnt=0;
39     int T; cin>>T;
40     while(T--){
41         printf("Case #d:\n",++cnt);
42         cin>>n;
43         for(int i=1;i<=n;i++) scanf("%lld",&a[i]);
44         Gauss();
45         int q; cin>>q;
46         while(q--){
47             ll k; scanf("%lld",&k);
48             printf("%lld\n", Query(k));
49         }
50     }
51 }

```